Parallel and Distributed R on Snowflake

This vignette outlines **parallel and distributed R** options in `snowflakeR`: the `foreach` backend **doSnowflake**, the **generic SPCS R executor**, and optional `crew` / **mirai** workers. All patterns assume you are connected with `sfr_connect()` and have the right Snowflake entitlements (SPCS compute pools, image repositories, and where applicable Hybrid Tables or Tasks).

For runnable Workspace examples, use the notebooks shipped under `system.file("notebooks", package = "snowflakeR")`, especially `workspace_parallel_spcs_setup.ipynb`, `workspace_parallel_spcs_demo.ipynb`, `snowflaker_parallel_spcs_config.yaml`, and `PARALLEL_SPCS_DEMO.md`.

doSnowflake (`foreach`)

Install the **Suggests** packages `foreach` and `iterators`, then register a backend:

```
library(snowflakeR)
library(foreach)

conn <- sfr_connect()

# In-container socket cluster (no SPCS required)
registerDoSnowflake(conn, mode = "local", workers = 4L)

out <- foreach(i = 1:20, .combine = c, .packages = "stats") %dopar% {
  median(rnorm(1000))
}
stopDoSnowflake()
```

Remote modes (SPCS + stage / queue infrastructure):

- `mode = "tasks"` – Snowflake Tasks orchestrate work; iterations are chunked into SPCS jobs. Supply `compute_pool`, `image_uri`, and optionally `stage`, `timeout_min`, `poll_sec`, `chunks_per_job` in `...`. Prepare the stage and validate pool/repo with `sfr_dosnowflake_setup()`.
- `mode = "queue"` – A Hybrid Table queue holds work items; SPCS workers (ephemeral or persistent) claim rows. Extra arguments include `n_workers`, `queue_fqn`, `worker_type`, `pre_warm`, `instance_family`, etc. See `?registerDoSnowflake`.
- `mode = "spcs"` – Not implemented yet (reserved).

```
sfr_dosnowflake_setup(conn,
  compute_pool = "MY_POOL",
  image_repo   = "MY_REPO"
)

registerDoSnowflake(conn, mode = "tasks",
  compute_pool = "MY_POOL",
  image_uri    = "mydb.myschema.myrepo/worker:latest"
)

# foreach body runs on workers ...
```

Use `?sfr_dosnowflake_build_image` when you need to build or refresh worker images for these modes.

Generic R executor (stage-mounted scripts)

For **ad hoc R jobs** on SPCS without `foreach`, upload an R script to a stage and run it via the executor bridge:

```
stage_path <- sfr_upload_r_script(conn, "analysis.R", stage = "MY_STAGE")

job <- sfr_execute_r_script(
  conn,
  r_script_stage_path = stage_path,
  compute_pool        = "MY_POOL",
  image_uri           = "mydb.myschema.myrepo/executor:latest",
  config              = list(region = "EMEA"),
  replicas            = 1L
)

sfr_executor_status(conn, job$job_name)
sfr_collect_executor_results(conn, job, stage = "MY_STAGE")
```

The script should define an entry function (default `main(config)`). See `?sfr_upload_r_script`, `?sfr_execute_r_script`, and `?sfr_collect_executor_results`.

crew + mirai on SPCS

For **long-lived controllers** and task queues using the **crew** ecosystem, `crew_controller_spcs()` configures workers as SPCS job services that connect back to the controller. This path requires suggested packages **crew** and **mirai** (and network/EAI setup appropriate for your account).

```
library(crew)
library(snowflakeR)

conn <- sfr_connect()
ctl <- crew_controller_spcs(
  conn,
  compute_pool = "MY_POOL",
  image        = "mydb.myschema.myrepo/r_crew:latest",
  workers      = 4L
)
ctl$start()
# push / pop tasks ...
ctl$terminate()
```

For a controller **running as an SPCS service**, see `?sfr_crew_controller_service` and `?sfr_crew_launch_workers`.

Choosing a pattern

Need	Typical API
Parallel `for` loops in R	`registerDoSnowflake()` + `%dopar%`
Snowflake-orchestrated chunks	`registerDoSnowflake(..., mode = "tasks")`
Shared queue + worker pool	`registerDoSnowflake(..., mode = "queue")`
One-off R script on SPCS	`sfr_upload_r_script()` + `sfr_execute_r_script()`
crew/mirai DAGs and workers	`crew_controller_spcs()`

Further reading

- **Function reference:** `?registerDoSnowflake`, `?sfr_dosnowflake_setup`, `?crew_controller_spcs`, `?sfr_execute_r_script`.
- **Internal design notes** (engineering detail): not shipped with the installed package; in the development repository under `internal/doSnowflake/`.